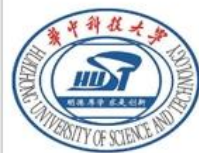


软件工程

第4章 软件设计

4.2 体系结构设计(第1部分: 体系结构的初步设计[续])



回顾: 如何设计软件体系结构?

辨识关键需求(关键的功能需求和关键的质量需求)

创建体系结构雏形

精化体系结构

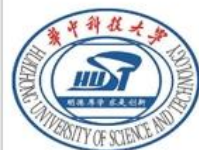
验证体系结构

创建体系结构雏形的基本方法

基于关键需求集，结合自身设计经验，选取合适的**体系结构模式**

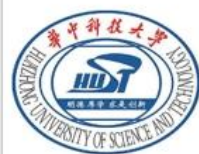
.....

什么是体系结构模式？



体系结构模式

- 规范地表示软件体系结构的设计经验，以支持其复用
- 体系结构模式的通常表示：
 1. 模式名
 2. 设计场景及(待解决的)设计问题
 3. 解决方案（体系结构及动态协作行为）
 4. 适用条件
 5. 正面效应
 6. 负面效应



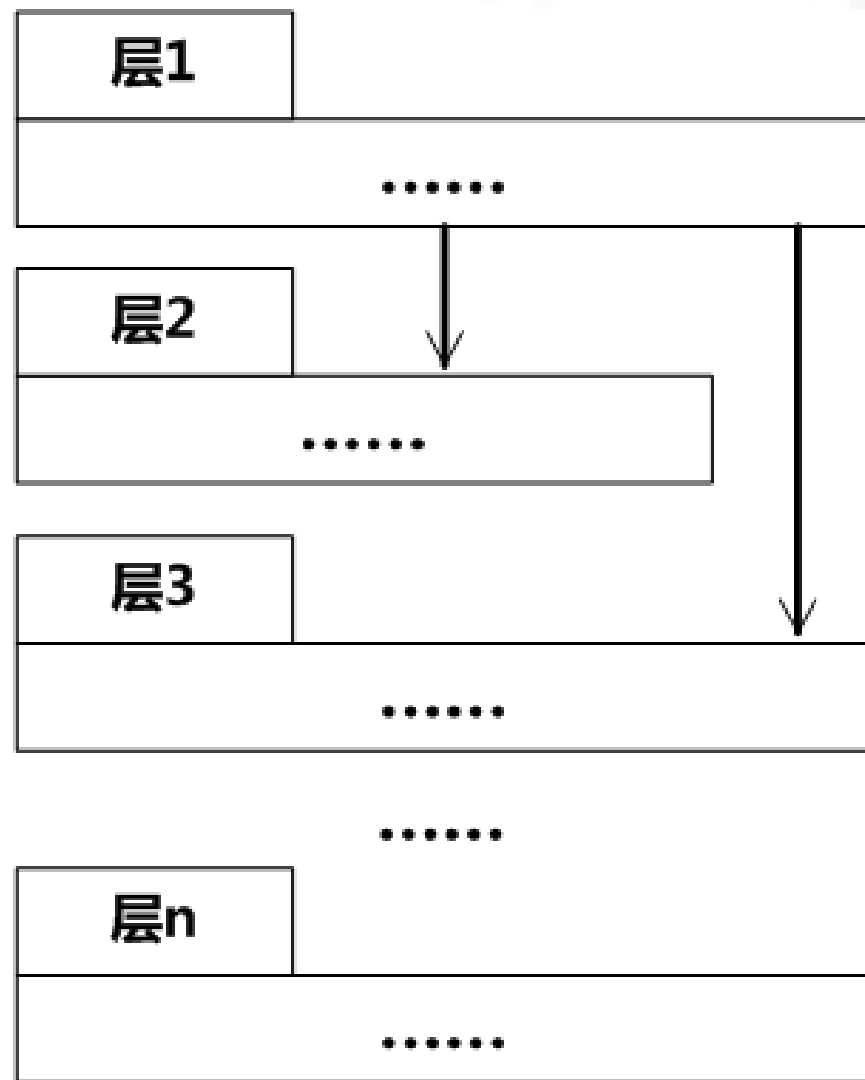
体系结构模式示例

1. 模式名：分层模式

2. 设计场景及问题：

- ☑ 目标软件系统需要分解；
- ☑ 系统中包含抽象级别明显不同的职责；高层职责的实现依赖于低层职责

3. 体系结构示意图



分层模式示意图的解释性说明(1)

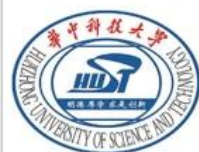
- 将软件系统按照抽象级别逐次递减的顺序划分为若干层，每层由若干抽象级别相同的模块组成
 - ◆ 在严格的分层结构中，每层的模块仅为近邻高层提供服务
 - ◆ 在稍宽松的分层结构中，服务提供者和接受者可跨越中间层，但前者一定位于更低的抽象层次



分层模式示意图的解释性说明(2)

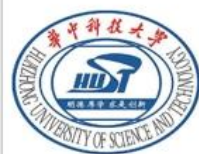
□ 一般而言

- ◆ 顶层直接面向用户提供软件系统的交互界面
- ◆ 底层负责提供基础性、公共性的技术服务，比较接近于运行环境（操作系统、数据库管理系统等）或物理设备接口
- ◆ 中间层的抽象级别介乎二者之间



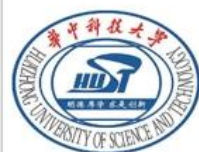
分层模式的动态协作过程(1/2)

- 高层模块向低层模块发出服务请求；
- 低层模块在计算完成后向请求者发送服务应答
 - ◆ 在计算过程中，低层模块可能向更低层模块发送抽象级别更低、粒度更细的服务请求



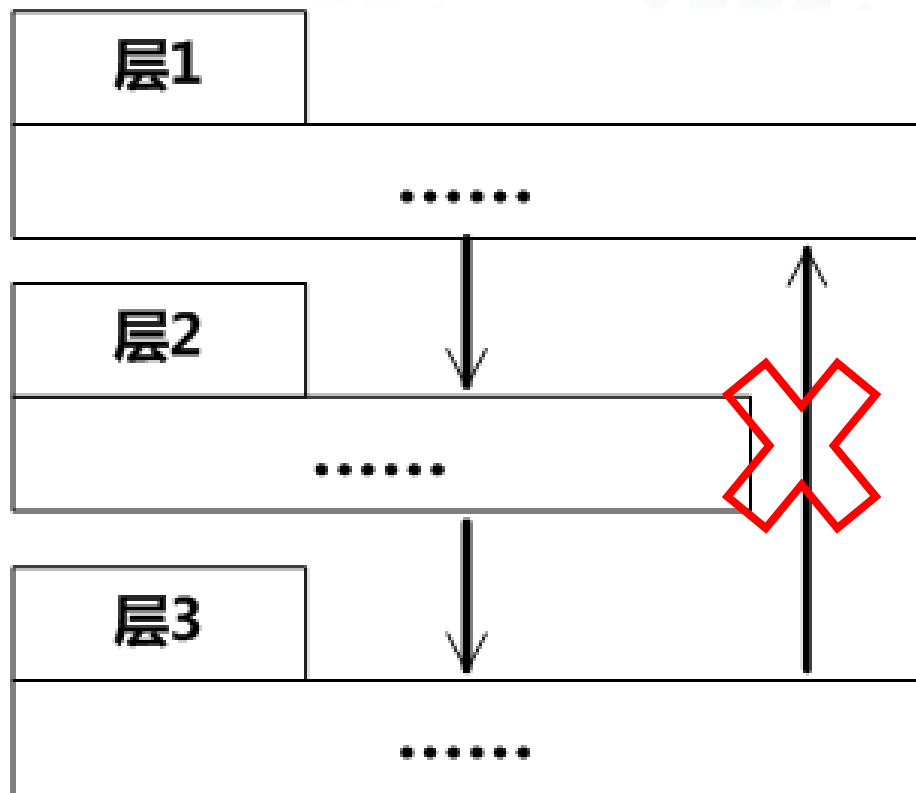
分层模式的动态协作过程(2/2)

- 低层模块在主动探测或被动获知计算环境的变化事件后通知高层
 - ◆ 通知链可一直延伸到最高层以便软件系统向用户报告，也可能中止于某个中间层



分层模式的适用条件

所有能够按不同抽象级划分为多层，并且服务请求/服务提供关系不会形成回路的软件系统



分层模式的正面效应

□ 松耦合：

- ◆ 通过层次划分和层间接口的规整，降低系统的耦合度，强化软件系统各模块之间依赖关系的局部化程度

□ 可替换性：

- ◆ 一个层次可以被实现了同样的对外服务接口的层次所替换
- ◆ 即便接口有所变化，层次替换的影响传播范围也仅限于直接使用该层服务的上层模块

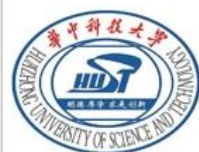
分层模式的正面效应(续)

□ 可复用性:

- ◆ 具有良好定义的服务请求接口的层次可在不同的上下文环境中实现复用

□ 标准化:

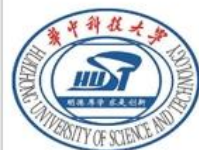
- ◆ 例如 ISO七层网络协议模型、POSIX接口标准、Java虚拟机



分层模式的负面效应

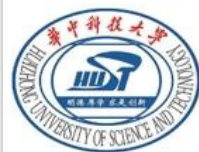
□ 性能开销：

- ◆ 高层功能可能需逐层调用下层服务，返回值、报错信息又需逐级上传，比直接实现高层功能更耗时
- ◆ 如果低层服务还完成了最初的服务请求者所不需要的冗余功能（为了给上层其他模块提供服务），性能损耗会更严重

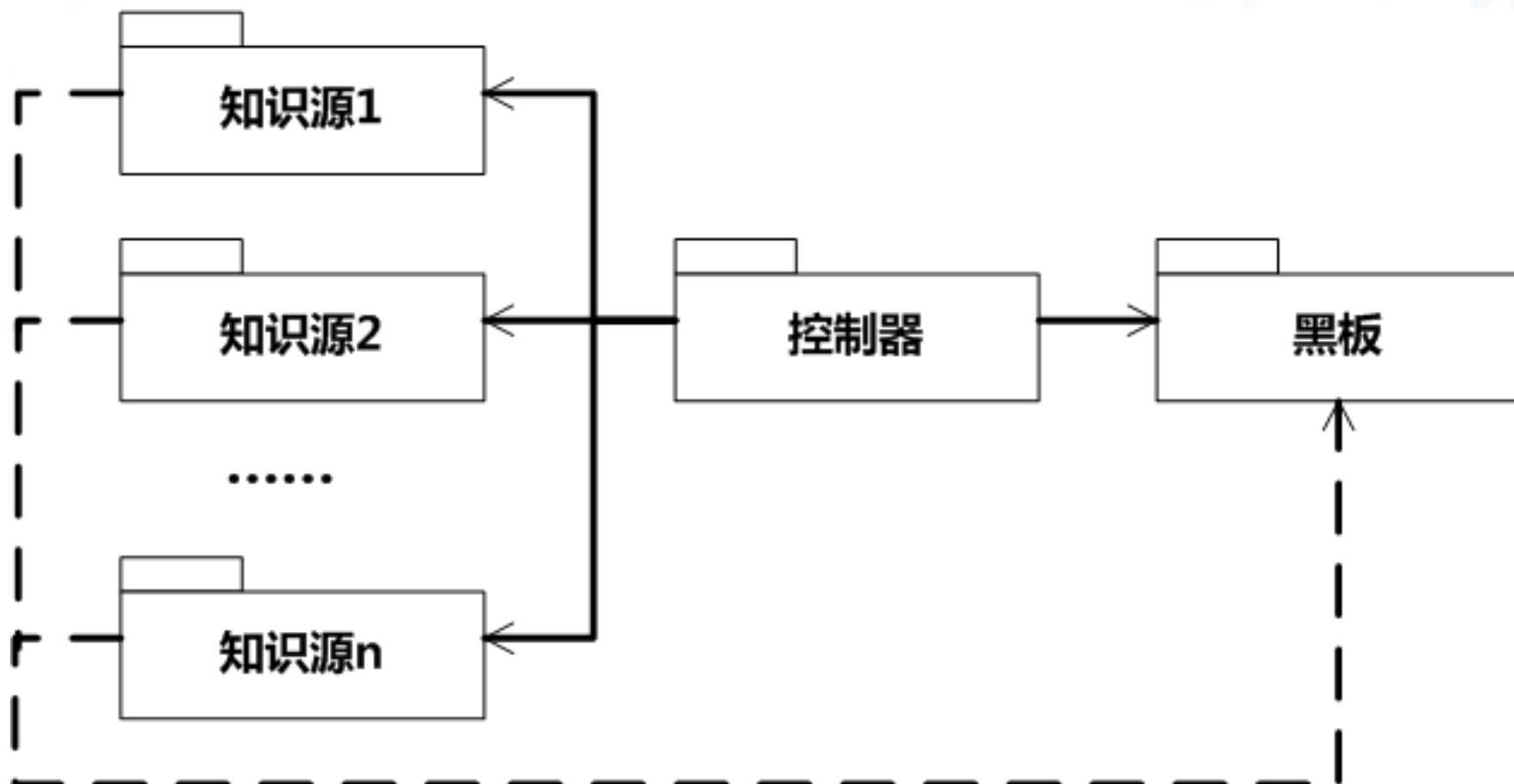


体系结构模式示例2

1. 模式名：黑板模式
2. 设计场景及问题：
 - ☑ 没有相近的已知或可行的解决方案；
 - ☑ 对解空间的完全搜索不可行；
 - ☑ 需要对同一子任务尝试不同的求解算法



黑板模式的示意图



黑板模式示意图的解释性说明(1)

□ 将系统划分为黑板、知识源和控制器三类模块

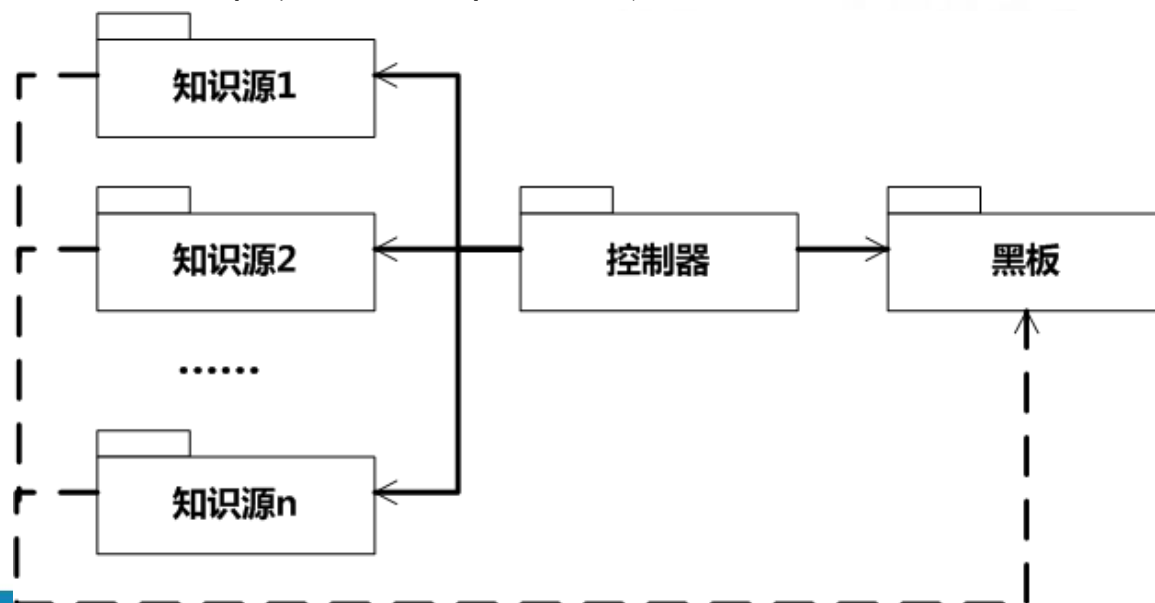
- ◆ 黑板负责保存问题求解过程中的状态数据，并提供读写服务
- ◆ 控制器负责监视黑板中不断更新的状态数据，安排知识源的活动
- ◆ 知识源负责根据黑板中存储的问题求解状态评价其自身的可应用性，进行部分问题求解工作，并将工作结果数据写入黑板



黑板模式的动态协作过程

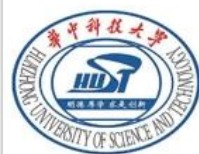
采用黑板模式的软件系统的典型动作过程如下：

- ◆ 控制器通过观察黑板状态数据决定哪些知识源对后续的问题求解可能有所贡献，然后调用这些知识源的评价功能以选取参与下一步求解活动的知识源；
- ◆ 被选中的知识源基于黑板状态数据将问题求解工作向前推进一步，并根据此步骤的结果更新黑板状态数据；
- ◆ 控制器不断重复上述控制过程，直至获得满意或比较满意的结果。



黑板模式的适用条件

黑板模式适合于没有确定的求解方法的
复杂问题



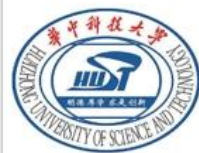
黑板模式的正面效应

□ 可扩展：

- ◆ 知识源和控制构件可灵活更换、升级，支持采用不同的知识源、不同的控制算法来试验各种问题求解方法

□ 可复用：

- ◆ 知识源之间没有互操作，知识源与控制构件和黑板之间均通过良好定义的接口进行交互，知识源的复用性较好



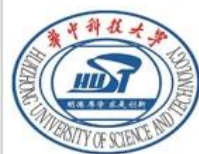
黑板模式的正面效应(2)

□ 容错性和健壮性：

- ◆ 知识源的每个问题求解动作都是探索性的，允许失败和试错

黑板模式的负面效应

- ❑ 问题求解性能较低，有时甚至无法预测求解时间
- ❑ 不能确保获得最优解
- ❑ 知识源和控制器两种模块的开发相当困难
- ❑ 问题求解路径不确定，造成软件测试困难



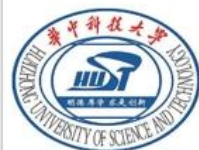
案例分析：针对MCS关键需求集， 选用什么模式？

□ 功能需求：

- ◆ 创建漫游环境
- ◆ 观察漫游过程
- ◆ 移动至指定位置[避障]

□ 质量需求：

- ◆ 可扩展性需求
- ◆ 可替换性需求



选用分层模式的理由

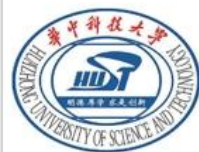
- MCS可自然地划分为界面、核心业务逻辑、接口三个层次，并且不会形成回路
- 根据可扩展性需求，必须将界面与核心业务逻辑分离，以支持从三种界面访问相同的业务逻辑处理模块
- 根据可替换性需求，必须将 接口 与 界面、核心业务逻辑 分离



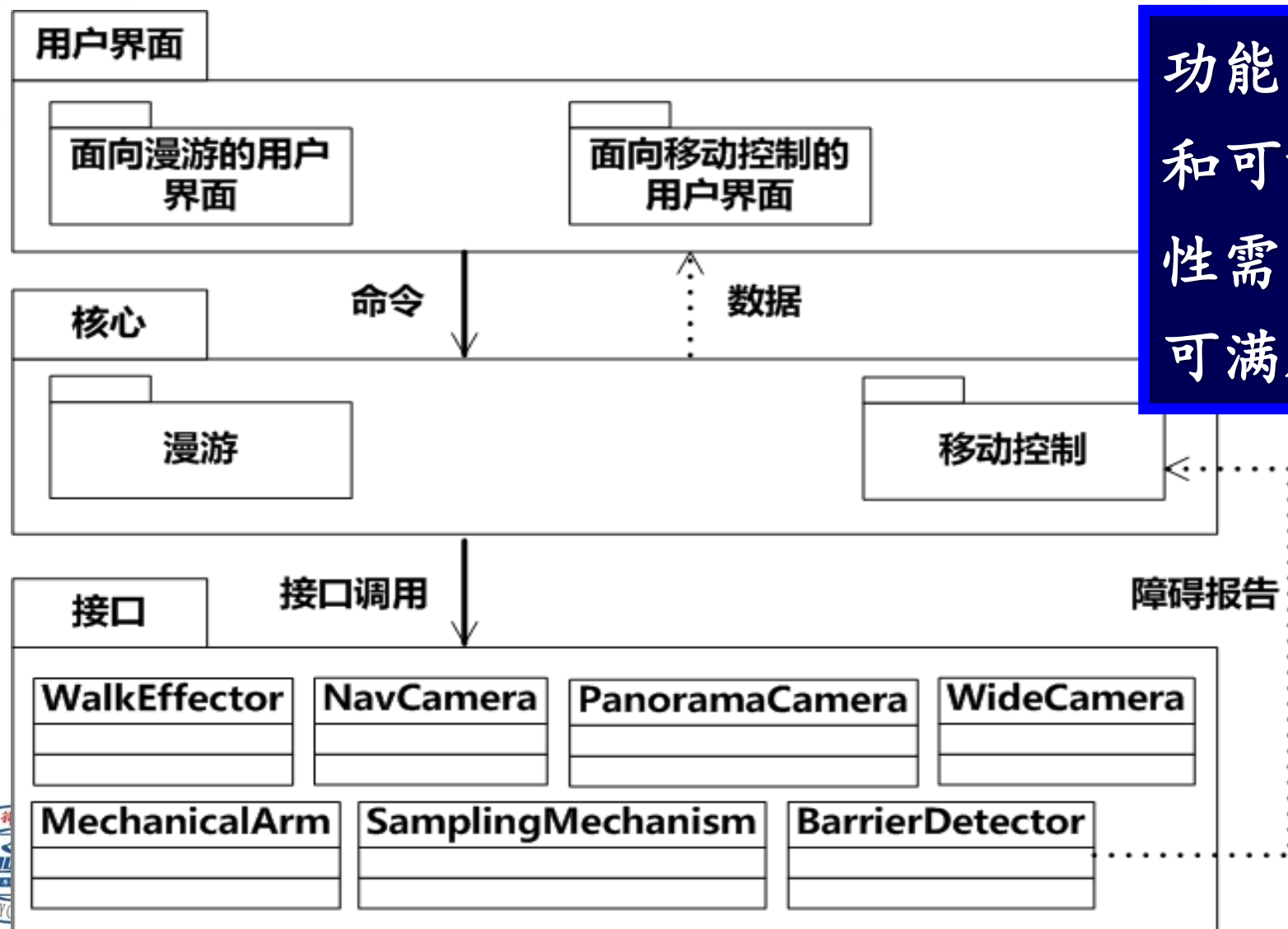
继续前进：创建体系结构雏形

基于关键需求集，结合自身设计经验，选取合适的体系结构模式

将分析模型中的职责注入到模式中合适的模块，并根据职责协作关系建立体系结构中这些模块之间的逻辑关系

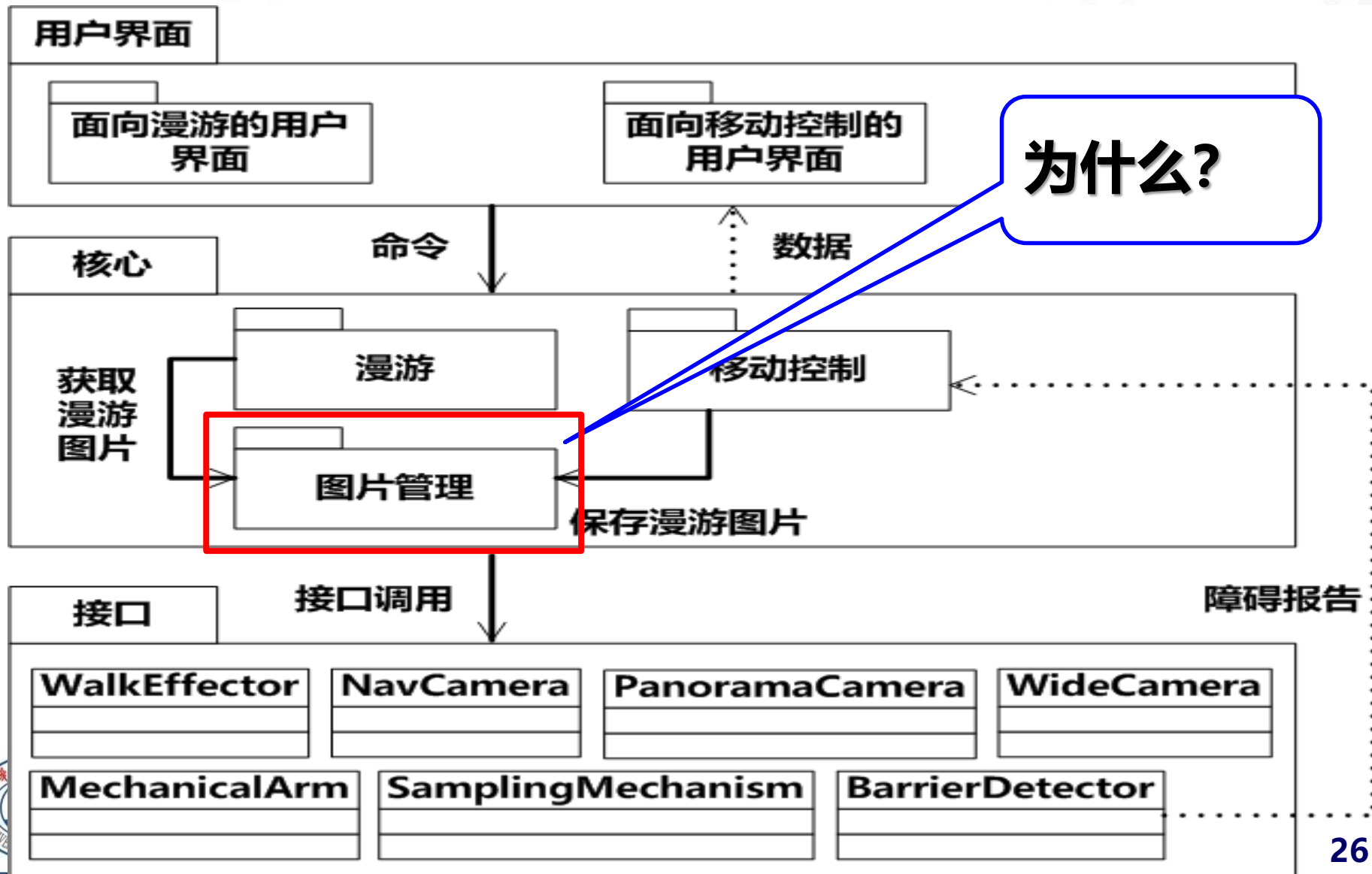


案例分析：MCS的体系结构雏形(第1步)



功能需求
和可替换
性需求已
可满足

第2步：如何满足可扩展性需求？



“图片管理”模块的职责设计

为未来支持大量爱好者通过Internet和智能手机查看漫游过程，

- 图片预取 + 内存cache管理
- 图片清晰度分级管理

“图片管理”模块的接口设计

```
public interface IImagePoolManager {  
    void saveImg(pathId, nSeqNo, img);  
    Image getImg(pathId, nSeqNo, nImgLevel);  
  
    void setup(cacheSize, cImgLevels);  
    void resizeCache(cacheSize);  
    void clearPool();  
    void release();  
}
```



总结：如何设计软件体系结构？

辩识关键需求(关键的功能需求和关键的质量需求)

创建体系结构雏形(选取适当的模式，注入分析模型中的职责，建立逻辑关联)

精化体系结构

且听下回分解

验证体系结构